

## Mid term ML

### Contribution Journal

I contributed to the midterm project by completing the section focused on preparing the training data for machine learning. My work included cleaning repeated categorical values, such as changing “No internet service” and “No phone service” into a simpler “No” value so the data was more consistent. I also one-hot encoded the categorical columns so the model could use them as numeric inputs, and he applied StandardScaler to the numerical columns tenure, MonthlyCharges, and TotalCharges. I also created before-and-after distribution plots to show how the data looked before and after scaling. In addition, I included a Logistic Regression model using class\_weight='balanced' to help address the class imbalance between customers who churned and customers who did not churn. Overall, my contribution helped make the training data cleaner, numeric, scaled, and more ready for future machine learning model building.

# \*\*Nigel Bontiff - Encoding, Scaling, and Distribution Comparison\*\*

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Preprocessing for the training data
train_person3 = train_df.copy()
print("Training data shape before Person 3 preprocessing:")
print(train_person3.shape)
print("\nFirst five rows before Person 3 preprocessing:")
print(train_person3.head())
```

### Repeated Categorical Values

```
object_columns = train_person3.select_dtypes(include=['object']).columns
for column in object_columns:
    train_person3[column] = train_person3[column].astype(str).str.strip()
service_columns = [
    'MultipleLines',
    'OnlineSecurity',
    'OnlineBackup',
    'DeviceProtection',
    'TechSupport',
    'StreamingTV',
    'StreamingMovies'
]
for column in service_columns:
    if column in train_person3.columns:
        train_person3[column] = train_person3[column].replace({
```

```

        'No internet service': 'No',
        'No phone service': 'No'
    })
print("Unique values after simplifying service-related columns:\n")
for column in service_columns:
    if column in train_person3.columns:
        print(column, ":", train_person3[column].unique())

```

## Features and Target

```

_train_person3 = train_person3.drop(columns=['Churn'])
y_train_person3 = train_person3['Churn']
# Convert Churn from Yes/No into 1/0
# Yes = customer churned
# No = customer did not churn
y_train_person3 = y_train_person3.map({
    'No': 0,
    'Yes': 1
})
print("Target value counts after converting Churn to 0 and 1:")
print(y_train_person3.value_counts())
print("\nTarget percentage distribution:")
print((y_train_person3.value_counts(normalize=True) * 100).round(2))

```

## One-Hot Encode Categorical Columns

```

# Find remaining categorical columns that still contain text
categorical_columns = X_train_person3.select_dtypes(include=['object']).columns
print("Categorical columns before one-hot encoding:")
print(list(categorical_columns))
# One-hot encode categorical columns
X_train_encoded = pd.get_dummies(
    X_train_person3,
    columns=categorical_columns,
    drop_first=True
)
# This section will convert True/False columns into 1/0 integers
bool_columns = X_train_encoded.select_dtypes(include=['bool']).columns
X_train_encoded[bool_columns] = X_train_encoded[bool_columns].astype(int)
print("\nTraining data shape after one-hot encoding:")
print(X_train_encoded.shape)
print("\nFirst five rows after one-hot encoding:")
print(X_train_encoded.head())

```

## Scaling Distributions

```
# The numeric columns I'm scaling
columns_to_scale = ['tenure', 'MonthlyCharges', 'TotalCharges']
# Show distributions before scaling.
plt.figure(figsize=(15, 4))
for i, column in enumerate(columns_to_scale):
    plt.subplot(1, 3, i + 1)
    plt.hist(X_train_encoded[column], bins=30)
    plt.title(f'{column} Before Scaling')
    plt.xlabel(column)
    plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

## StandardScaler Section

```
# StandardScaler to put numeric columns on a similar scale
scaler = StandardScaler()
X_train_scaled = X_train_encoded.copy()
X_train_scaled[columns_to_scale] = scaler.fit_transform(
    X_train_scaled[columns_to_scale]
)
print("First five rows after scaling tenure, MonthlyCharges, and TotalCharges:")
print(X_train_scaled[columns_to_scale].head())
print("\nSummary statistics after scaling:")
print(X_train_scaled[columns_to_scale].describe())
```

## After Scaling Distributions Section

```
# display for distributions after scaling
plt.figure(figsize=(15, 4))
for i, column in enumerate(columns_to_scale):
    plt.subplot(1, 3, i + 1)
    plt.hist(X_train_scaled[column], bins=30)
    plt.title(f'{column} After Scaling')
    plt.xlabel(column)
    plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

## Logistic Regression with Class Balancing Section

```
# I created a validation split from the training data so the original test_df stays untouched
```

```

X = final_train_person3.drop(columns=['Churn'])
y = final_train_person3['Churn']
X_train_model, X_val_model, y_train_model, y_val_model = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
log_reg_model = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42
)
log_reg_model.fit(X_train_model, y_train_model)
y_pred = log_reg_model.predict(X_val_model)
print("Accuracy Score:")
print(accuracy_score(y_val_model, y_pred))
print("\nClassification Report:")
print(classification_report(y_val_model, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_val_model, y_pred))

```

# Discuss how you would use this dataset to solve a ML problem

- This preprocessing helps because the original Telco Churn dataset has a lot of text values that a machine learning model cannot use by itself. For example, columns like internet service, payment method, contract type, and other customer service features are written as words instead of numbers. By cleaning values like “No internet service” and “No phone service” into just “No,” the data becomes more consistent and easier to work with.

After that, one-hot encoding changes the categorical columns into 0 and 1 values, which makes the data usable for a machine learning model. I also scaled the numerical columns, including tenure, MonthlyCharges, and TotalCharges, because those columns are on different ranges. Scaling helps make sure one feature does not seem more important just because it has larger numbers.

I also used `class_weight='balanced'` in the Logistic Regression model because the dataset has more customers who did not churn than customers who did churn. This helps the model pay more attention to the smaller churn group instead of mainly predicting the majority class. Overall, these steps help turn the training data into a cleaner and more model-ready dataset that could be used to predict whether a customer is likely to churn.